LAPORAN PRAKTIKUM
WEB DESAIN
MODUL 2

Dosen Pengampu: Dede Irawan, M.Kom.



Disusun oleh :

Nama	**:**	**Dina Nurrahmayanti**
NIM	**:**	**20250910041**
Kelas	**:**	**SINFC-02-2025**

FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke Hadirat Allah SWT atas rahmat dan karunia-Nya, sehingga laporan Praktikum Desain Web ini dapat diselesaikan dengan baik.

Laporan ini disusun guna memenuhi tugas mata kuliah Praktikum Desain Web di Fakultas Ilmu Komputer, Universitas Kuningan. Di dalamnya, berisi rangkaian modul pembelajaran yang telah dipelajari.

Penulis menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi penulis dan pembaca.

Kuningan, 29 Oktober 2026

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I 4
PRETEST **4**
BAB II 5
PRAKTIKUM **5**
BAB III 7
POSTEST **7**
BAB IV 8
TUGAS/LATIHAN **8**

BAB I

PRETEST

1. Apa tujuan dari menggunakan version control dalam pengembangan perangkat lunak?
   Version control mencatat setiap perubahan pada proyek sehingga riwayatnya bisa dilacak dan dikembalikan ke versi sebelumnya bila terjadi kesalahan. Ia juga memungkinkan banyak orang mengerjakan proyek yang sama tanpa saling menimpa, karena setiap perubahan tercatat siapa, kapan, dan mengapa. Bila disimpan di remote repository seperti GitHub, ia sekaligus menjadi cadangan proyek.
2. Jelaskan perbedaan antara git add dan git commit!
   git add memindahkan perubahan dari working directory ke staging area, yaitu tahap memilih perubahan yang akan direkam. git commit merekam isi staging area menjadi snapshot permanen di repository lokal, lengkap dengan pesan, penulis, dan waktu. Singkatnya: git add memilih, git commit menyimpan.
3. Bagaimana cara membuat branch baru di Git?
   Gunakan git branch nama-branch, lalu pindah dengan git checkout nama-branch atau git switch nama-branch. Keduanya bisa digabung: git checkout -b nama-branch atau git switch -c nama-branch. Daftar branch dilihat dengan git branch.

BAB II

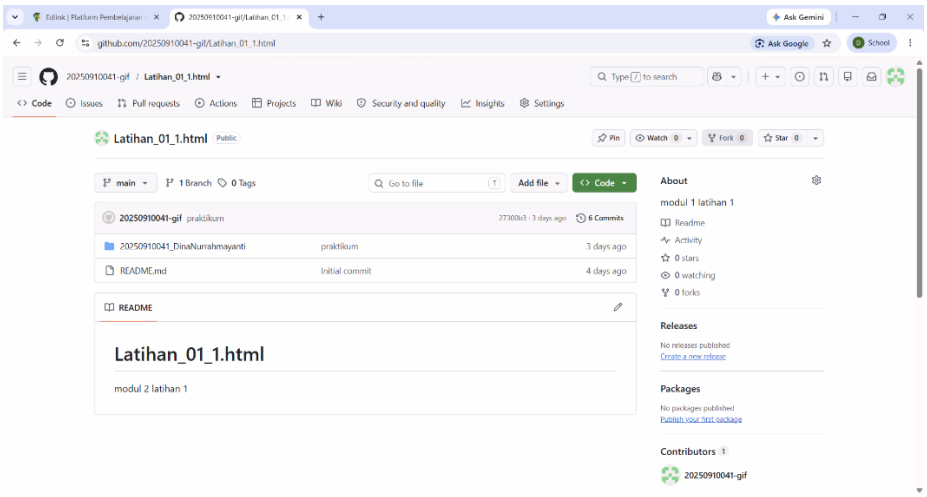
PRAKTIKUM

1. Membuat Akun GitHub
    - Saya membuka https://github.com/ di Google Chrome dan menekan Sign up di pojok kanan atas.
    - Lalu saya mengisi email aktif, kata sandi yang kuat, dan username. Setelah muncul centang hijau tanda username tersedia, saya menekan Continue.
    - GitHub lalu meminta verifikasi. Saya menekan Verify dan menyelesaikan teka-teki sesuai yang diberikan, lalu Submit.
    - Setelah itu kode verifikasi masuk ke email saya. Dan muncul halaman Sign in to GitHub dengan pesan Your account was created successfully. Saya login, dan melengkapi profil.
2. Instalasi Git
    - Saya mengunduh Git dari https://git-scm.com/, menjalankan installer, dan mengikuti wizard dengan pilihan default (Next, Install, Finish). Untuk memastikan berhasil, saya membuka Command Prompt dan mengetik: `git --version`
    - Terminal menampilkan nomor versi Git, tanda instalasi berhasil.
    - Selanjutnya saya membuka Visual Studio Code, memilih File, lalu klik Open Folder untuk membuka folder kerja saya, lalu membuka terminal dengan Ctrl + ` (atau View klik Terminal). Di sana saya mengatur identitas pengguna, yang akan tercatat di setiap commit:
      `git config --global user.name "20250910041-gif"`
      `git config --global user.email "20250910041@uniku.ac.id"`
    - Saya memakai email yang sama dengan akun GitHub dan memeriksanya dengan git config --list.
3. Mengelola Proyek dengan Git
    - Pada terminal VS Code, saya menginisialisasi repository di folder kerja:
      `git init`
    - Git membuat folder tersembunyi .git yang menyimpan seluruh riwayat proyek. Berikutnya saya memasukkan berkas latihan Modul 1 ke staging area:
      `git add 20250910041_DinaNurrahmayanti/Modul_01/Latihan_01_01.html`
    - Perintah itu tidak menampilkan pesan, jadi saya memeriksanya:
      `git status`
    - Berkas muncul hijau di bagian Changes to be committed sebagai new file, artinya siap di-commit. Berkas yang belum di-add muncul merah di Untracked files. Lalu saya merekamnya:
      `git commit -m "Initial commit: menambahkan file Latihan_01_01.html"`
    - Git menampilkan ringkasan commit. Saat saya jalankan git status lagi, muncul nothing to commit, working tree clean, dan git log --oneline menampilkan commit pertama saya.

4. Menghubungkan ke GitHub dan Push Perubahan

- Di GitHub saya mengklik New repository, mengisi nama repository Latihan_01_1.html, memilih Public, lalu Create repository. Repository ini sudah berisi README.md dari Initial commit. GitHub menampilkan URL repository saya.
- Di terminal VS Code, saya menghubungkan repository lokal dengan remote, menyeragamkan nama branch utama, lalu mengirim commit:
  git remote add origin https://github.com/20250910041-gif/Latihan_01_1.html.git
  git branch -M main
  git pull origin main --allow-unrelated-histories
  git push -u origin main
- Baris git pull diperlukan karena repository di GitHub sudah memiliki README.md, sehingga riwayat lokal dan remote perlu disatukan dulu sebelum push. Saat push pertama, Git Credential Manager meminta login; saya memilih Sign in with your browser dan menyetujui otorisasi (GitHub tidak lagi menerima kata sandi biasa untuk push via terminal). Terminal lalu menampilkan proses pengiriman dan baris branch 'main' set up to track 'origin/main'. Opsi -u membuat git push berikutnya cukup diketik tanpa tambahan.

5. Mengamati Hasil
   - Setelah me-refresh halaman repository, folder 20250910041_DinaNurrahmayanti muncul berdampingan dengan README.md, lengkap dengan pesan commit dan waktunya. Isi berkas di GitHub sama dengan yang ada di komputer saya. Dari sini saya melihat bahwa proyek kini tersimpan di cloud, riwayat commit tercatat di tab Commits.



BAB III

POSTEST

1. Apa fungsi dari git merge dalam workflow Git?
   git merge menggabungkan perubahan dari satu branch ke branch lain, biasanya hasil kerja branch fitur ke main. Fitur dapat dikembangkan terpisah tanpa mengganggu kode utama, lalu digabung setelah selesai. Contoh: git switch main, lalu git merge fitur-baru.
2. Jelaskan cara mengatasi konflik saat melakukan merge!
   Konflik terjadi ketika dua branch mengubah baris yang sama pada berkas yang sama. Cara mengatasinya:
   1. Jalankan git status untuk melihat berkas yang berstatus both modified.
   2. Buka berkas itu. Bagian konflik ditandai seperti berikut: <<<<<<< HEAD
   3. versi dari branch saat ini (main) =======
   4. versi dari branch yang digabung (fitur-baru) >>>>>>> fitur-baru
   5. Pilih salah satu versi atau gabungkan keduanya, lalu hapus penanda <<<<<<<, =======, dan >>>>>>>.
   6. Simpan, jalankan git add nama-berkas, lalu git commit (atau git merge --continue).
   7. Untuk membatalkan merge, gunakan git merge --abort.
3. Tulis langkah-langkah untuk menambahkan file baru ke repository Git dan melakukan commit.
   1. Simpan berkas baru di folder proyek, misalnya Modul_03/Latihan_03_01.html.
   2. Cek dengan git status (berkas berstatus untracked).
   3. Masukkan ke staging area: git add Modul_03/Latihan_03_01.html (atau git add . untuk semua berkas).
   4. Rekam: git commit -m "Menambahkan Latihan_03_01.html".
   5. Kirim ke GitHub (opsional): git push.

BAB IV

TUGAS/LATIHAN

1. Buatlah repository Git untuk proyek web dan dokumentasikan proses pembuatan, commit, dan push ke GitHub.
   Saya memakai repository Latihan_01_1.html sebagai proyek web saya
   - Pembuatan repository
     Perintah: git init
     Hasil yang diamati: Folder .git terbentuk.
   - Menambah berkas
     Perintah: git add 20250910041_DinaNurrahmayanti/Modul_01/
     Hasil yang diamati: Berkas masuk staging area.
   - Commit
     Perintah: git commit -m "praktikum"
     Hasil yang diamati: Commit baru tercatat.
   - Menghubungkan remote
     Perintah: git remote add origin https://github.com/20250910041-gif/Latihan_01_1.html.git
     Hasil yang diamati: Remote origin terdaftar.
   - Push
     Perintah: git push -u origin main
     Hasil yang diamati: Commit tampil di GitHub.
2. Rancanglah workflow Git yang melibatkan branch untuk fitur baru dan integrasi dengan branch utama.
   Workflow Git dilakukan dengan mengembangkan fitur pada branch terpisah agar main tetap stabil.
   1. Perbarui branch utama: git switch main dan git pull.
   2. Buat branch fitur: git switch -c fitur-navbar.
   3. Tambahkan dan commit perubahan menggunakan git add dan git commit.
   4. Kirim branch: git push -u origin fitur-navbar.
   5. Buat Pull Request ke main melalui GitHub (opsional).
   6. Gabungkan branch fitur:
      git switch main
      git merge fitur-navbar
      git push
3. Simpan seluruh praktikum pada modul sebelumnya kedalam repository Git yang sudah anda buat
   Saya menyimpan seluruh berkas Modul 1 ke repository menggunakan perintah:
   git status
   git add .
   git commit -m "praktikum"
   git push